

Product Requirements Document (PRD)

Project: Tabular RAG Assistant (Heart Disease Dataset)

Context: AI Internship Assignment - TCS Indore

1. Executive Summary

Objective: To build a Retrieval-Augmented Generation (RAG) application capable of accurately answering user queries based *exclusively* on the provided `heart_dataset.csv`. **Key**

Constraints: * The system must use a 100% free local tech stack (HuggingFace, Chroma, Ollama).

- Token-to-vector conversion must be stored in Chroma.
- The system must strictly output exactly: **"Sorry I can't find anything relevant to this question"** for out-of-context queries.

2. Tech Stack Requirements

- **Orchestration:** LangChain (Python).
- **Vector Data Store:** Chroma (ChromaDB) running locally.
- **Embedding Model (Encoder):** `sentence-transformers/all-MiniLM-L6-v2` or `BAAI/bge-small-en-v1.5` (via Hugging Face).
- **Regressive Model (LLM):** **Ollama** running locally. Recommended models: `llama3` (8B) or `mistral` (7B) for high accuracy on contextual reasoning.
- **UI Framework:** Streamlit (recommended for fast iteration with tabular data) or React/FastAPI (if continuing the previous cinematic design).

3. Data Ingestion & Processing Strategy (Critical for CSVs)

Standard `RecursiveCharacterTextSplitter` will destroy the relationships between columns in a CSV. To minimize inference errors, the ingestion pipeline must serialize the tabular data semantically.

- **Parser:** Use LangChain's `CSVLoader`.
- **Semantic Row Conversion:** Customize the loader to parse each row into a natural language sentence rather than raw key-value pairs.
 - *Example transformation for Row 1:* "Patient is a 40 year old Male. Chest pain type is ATA. Resting blood pressure is 140. Cholesterol is 289. Fasting blood sugar is 0. Resting

ECG is Normal. Maximum heart rate is 172. Exercise angina is N. Oldpeak is 0. ST slope is Up. Heart disease diagnosis is 0."

- **Metadata Tagging:** Store the raw row data (Age, Sex, Cholesterol) as metadata in the Chroma document so the LLM has exact figures available upon retrieval.

4. Retrieval & Query Mechanism

- **Vectorization:** Pass the semantically converted rows through the free HuggingFace encoder to generate dense vectors, storing them in Chroma.
- **Retrieval Method:** Use Cosine Similarity search to retrieve the Top-K (e.g., K=5) patient records most semantically relevant to the user's query (e.g., "Tell me about young female patients with high cholesterol").

5. Error Minimization & Guardrails (Inference)

Tabular data is highly prone to LLM hallucinations (e.g., the LLM doing bad math or inventing patients). The following techniques are mandatory:

1. Pre-LLM Distance Thresholding (The Primary Guardrail):

- Before sending the prompt to Ollama, check the similarity distance of the top retrieved Chroma document.
- If the distance is above a strict threshold (e.g., > 0.5 , depending on the embedding model), the system must **bypass Ollama completely** and immediately return: *"Sorry I can't find anything relevant to this question"*. This guarantees 100% compliance for out-of-domain questions.

2. Strict System Prompting with Tabular Instruction:

- Configure the LangChain prompt template to restrict Ollama's behavior.
- *Template Example:*

```
You are a medical data assistant. Use ONLY the provided Patient Records to answer.
Do not make assumptions, calculate statistics unless explicitly clear, or
If the provided records do not contain the answer, you must reply EXACTLY
```

```
Patient Records Context: {context}
Question: {question}
```

3. Low Temperature Configuration:

- Set Ollama's temperature parameter to 0.0 . This removes all creative generation, forcing the model to act as a strict information extractor for the CSV records.

6. Expected Deliverables & Success Metrics

1. **Script/Application:** A functional Python script or web app demonstrating the end-to-end pipeline.
2. **Accuracy:** The system successfully extracts patient profiles matching specific queries without hallucinating unlisted traits.
3. **Guardrail Adherence:** 100% success rate in returning the exact fallback phrase when queried about non-heart dataset topics (e.g., "What is HIPAA?" or "Who won the World Cup?").
4. **Cost:** Architecture uses \$0.00, fully utilizing local hardware via Ollama and Chroma.